Intelligent Traffic Simulator:

This is inteloogent simulator, which mimics the the actuall traffic on road. I this simulator the user is given option to choose to its vehicle and its destination. Then it uses the DIsjistra's Alogotrithm to choose the closes path between the destination and the starting point. Moreover, everytime it finishes its destination it get some amount money as reward like as if it is a careem app. There is a refueling bar and station placed on the map , where it refuels itself.

GOAP IMPLEMENTATION:

`GoapAction`: This struct represents a single behavior. It contains a name, a numerical **cost**, a set of **preconditions** (what must be true to start), **effects** (what changes after finishing), and a functional **execution logic**.
`GoapNode`: Used during the planning phase to represent a potential state of the world. It tracks the current simulated state, the list of actions taken to get there, and the total accumulated cost.
Priority Queue: The system uses a priority queue to always explore the lowest-cost "nodes" first, ensuring the vehicle finds the most efficient plan.
Refueling: When car/bike has low fuel it changes it's destination to refueling station its car and then returns back to its destination.

HEADER FILES OOP IMPLEMENTATION:

1. common.h

- **Responsibility: This is your data foundation. It defines all the shared constants (like PI), enumerations (VehicleType, GameState), and fundamental structs (Waypoint, Obstacle). It ensures all other files speak the same "language."**
- **OOP Implementation: Data Abstraction. Instead of passing around loose x and y coordinates and random integers, you abstract them into meaningful objects (e.g., bundling position, rotation, and type into a BlockadeCar struct). This groups related data together logically.**

2. Pathfinding.h

- **Responsibility: It contains the Dijkstra algorithm logic. It takes a starting point, an end point, and a map, and returns the mathematically shortest path while ignoring blocked roads.**
- **OOP Implementation: Functional Abstraction.**
- **This file hides the messy, complicated math (priority queues, distance vectors, square roots) from the rest of the game. A Vehicle object doesn't need to know *how* Dijkstra works; it just calls CalculatePath() and gets an answer.**

3. GOAP.h

- **Responsibility: This is the AI "brain" engine. It defines what an action is (`GoapAction`) and contains the `BuildPlan` algorithm that searches for the best sequence of actions to reach a goal.**
- **OOP Implementation: Encapsulation.**
- **It encapsulates the concept of a decision into an object. A `GoapAction` holds its own rules (preconditions) and results (effects) tightly packaged together. The complexity of A* search planning is safely hidden inside the `BuildPlan` function.**

4. UIHelpers.h

- **Responsibility: Manages the visual user interface, specifically handling the logic for drawing interactive buttons, centering text, and detecting mouse hovers.**
- **OOP Implementation: Encapsulation & Reusability.**
- **It bundles Raylib's complex drawing commands into a single, clean `DrawButton()` interface. This means your main file doesn't need to repeat 10 lines of code to draw a hover-effect; it just calls the helper object.**

5. VehicleCore.h

- **Responsibility: Defines the main blueprint for a vehicle in the simulation. It holds the core state: is it a player, what type is it, how much fuel does it have, and where is it going?**
- **OOP Implementation: Encapsulation & Class Design.**
- **This is the most traditional OOP file. It acts as the primary Class, bundling the *attributes* (fuel, money, position) together. It protects the vehicle's internal state so that outside code cannot randomly break the vehicle's logic.**

6. VehicleDrive.h

- **Responsibility: Handles the physical physics and movement logic. It calculates speeds, updates coordinates based on the current path, handles traffic jams, and detects traffic lights.**
- **OOP Implementation: Separation of Concerns (Modularity).**
- **In a bad design, movement math would be crammed into the main core. By splitting it here, you are modularizing the vehicle's behaviors. It encapsulates the "driving" math so the core vehicle object doesn't get bloated.**

7. VehicleActions.h

- **Responsibility: Bridges the vehicle to the GOAP system. It creates the specific, concrete actions a vehicle can take (like "Drive to Destination" or "Detour to Gas Station").**
- **OOP Implementation: Behavioral Delegation (Polymorphism concept).**
- **Instead of writing a massive `if/else if/else` block to control the AI, you delegate behaviors into independent action objects. The vehicle can dynamically swap these behaviors at runtime without changing its core code.**

8. `VehicleDesign.h`

- **Responsibility: Strictly handles rendering. It draws the correct texture (sedan vs. bike), rotates the image to match the driving angle, draws the health/fuel bars, and handles the exhaust particle effects.**
- **OOP Implementation: Interface/View Separation.**
- **This file represents the "View" in object-oriented design. It separates the *visual representation* of the vehicle from the *logical calculations* of the vehicle. If you want to change the car from 2D to 3D later, you only have to rewrite this file, not the entire driving logic!**

WHO DID WHAT?

Team Responsibilities: Traffic Simulator Project

- ## Muhammad Umer Syed
  - **`GOAP.h`: Developed the Goal-Oriented Action Planning artificial intelligence and decision-making logic.**
  - **`VehicleActions.h`: Bridged the AI brain to specific vehicle behaviors and state management.**
  - **`VehicleDrive.h`: Implemented physical vehicle movement, speed calculations, and path execution.**
  - **`Traffic_Simulator.cpp` *(Co-authored)*: Assisted in integrating the AI and movement logic into the main simulation loop.**
- ## Rufile
  - **`Pathfinding.h`: Built the Dijkstra algorithm to calculate optimal routes across the city nodes while avoiding blockades.**
  - **`UIHelpers.h`: Managed the visual user interface, including interactive buttons, menus, and text rendering.**
  - **`Traffic_Simulator.cpp` *(Co-authored)*: Assisted in building the core execution loop and UI state management.**
- ## Emad
  - **`VehicleCore.h`: Designed the primary Vehicle class structure, state variables, and core attributes.**

    - **`VehicleDesign.h`: Handled the visual rendering of the simulation, including vehicle textures, scaling, and particle effects.**
    - **`common.h`: Defined the foundational data structures (Waypoints, Obstacles) and shared enumerations for the entire project.**
    - **`Traffic_Simulator.cpp` *(Co-authored)*: Assisted in tying the core data structures and rendering logic into the main application.**
- **Shared Responsibility (Entire Team)**
    - **Main Application (`Traffic_Simulator.cpp`): All three members collaborated on the main file to initialize the Raylib window, set up the simulation environment, manage the global game states (Menu, Gameplay, Game Over), and execute the primary loop.**